

1.SGLang 的 RadixAttention / RadixCache 解决什么问题？

是为了避免模型反复计算相同的内容。不同用户的问题中，可能有一部分输入是一样的。如果每次都从头开始计算，就会浪费很多计算资源。SGLang 会把之前已经计算过的内容对应的 KV Cache 保存下来。当新的请求进来时，就会检查它和之前的请求有没有相同的前缀。如果有，就可以直接使用之前保存的结果，不需要再重新计算。

2.page-sized KV cache 与 prefix reuse 分别对应 pipeline 中的哪一部分？

page-sized KV Cache 和 KV Cache 的存储和显存管理有关。它把 KV Cache 按照一定大小的块进行管理，这样不用给每个请求提前分配一整块连续的显存，可以根据实际需要进行分配，显存利用会更加灵活。

Prefix Reuse 主要对应 Prefill 之前的缓存查找过程。SGLang 会通过 RadixCache 检查当前请求有没有和之前请求相同的前缀。如果有，就可以直接复用之前的 KV Cache，减少 Prefill 阶段需要重新计算的内容。

3. vLLM 的 PagedAttention 解决什么问题？它和 SGLang 的 RadixAttention / paged KV 机制有什么联系与差异？

是为了解决 KV Cache 占用和管理显存的问题。PagedAttention 的思路是把 KV Cache 分成一个个固定大小的块，需要多少就使用多少。这样可以更灵活地分配显存，提高 GPU 显存的利用率，也方便同时处理多个请求。SGLang 的 RadixAttention / RadixCache 和 PagedAttention 关注的重点不完全一样。PagedAttention 更主要解决 KV Cache 的存储和显存分配问题，而 RadixAttention / RadixCache 更主要解决不同请求之间相同前缀的重复计算问题。